

Student 3 **DANIMIZA V** 192421363RC

2 / 2 submitted

Q1. Student Grade Book Application**CODE**

```
students = {"Alice": [95, 90, 88, 92, 96], "Bob": [78, 82, 75, 80, 96], "Charlie": [65, 70, 68, 72, 66], "David": [55, 60, 58, 62, 57], "Eva": [85, 88, 91, 87, 90]}

def get_grade(avg):
    if avg >= 90:
        return "O"
    elif avg >= 80:
        return "A+"
    elif avg >= 70:
        return "A"
    elif avg >= 60:
        return "B+"
    elif avg >= 50:
        return "B"
    else:
        return "F"

results = {name: {"marks": marks, "Total": sum(marks), "Average": sum(marks)/len(marks), "Grade": get_grade(sum(marks)/len(marks))} for name, marks in students.items()}

topper = max(results.items(), key = lambda x: x[1]["Average"])
num_subjects = 5
subject_means = [sum(students[name][i] for name in students)/len(students) for i in range(num_subjects)]
highest_subject = subject_means.index(max(subject_means))+1
sorted_results = sorted(results.items(), key = lambda x: x[1]["Average"], reverse = True)
print("\n" * 80)
for name, data in sorted_results:
    print("{:<10}{:<30}{:<8}{:<10.2f}{:<6}".format(name, str(data["marks"]), data["Total"], data["Average"], data["Grade"]))
print("\n" * 80)
print("\nclass topper:")
print(f"{topper[0]} with Average = {topper[1]['Average']:.2f}")
print("\nsubject means:")
for i, mean in enumerate(subject_means, start = 1):
    print(f"subject{i}:{mean:.2f}")
    print(f"\nsubject with highest overall mean: subject{highest_subject}")
```

OUTPUT (no output)

Q2. Debugging and Rewriting**CODE**

```
accounts = {}
def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name':name, 'balance':balance, 'txns':[]}
def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('deposit', amount))
    else:
        print("account not found")
def withdraw(acc_no, amount):
    if acc_no in accounts:
        if accounts[acc_no]['balance'] >= amount:
            accounts[acc_no]['balance'] -= amount
            accounts[acc_no]['txns'].append(('withdraw', amount))
        else:
            print("insufficient balance")
    else:
        print("account not found")
def mini_statement(acc_no):
    if acc_no in accounts:
        print("\nmini statement")
        for t in accounts[acc_no]['txns'][-5:]:
            print(t)
    else:
        print("account not found")
acc_no = int(input("enter account number:"))
name = input("enter name:")
balance = float(input("enter initial balance:"))
create_account(acc_no, name, balance)
deposit_name = float(input("enter deposit amount:"))
deposit(acc_no, deposit_name)
withdraw_amount1 = float(input("enter first withdrawal amount:"))
withdraw(acc_no, withdraw_amount1)
withdraw_amount2 = float(input("enter second withdrawal amount:"))
withdraw(acc_no, withdraw_amount2)
mini_statement(acc_no)
print("\nbalance:",accounts[acc_no]['balance'])
```

INPUT

```
1001
ravi
10000
5000
20000
3000
```

OUTPUT

```
(no output)
```